



Exercise 5

JAVA JSON

23127449 – Nguyễn Tấn Phát

LEARNING OBJECTIVES

01

Learn the structure and syntax of JSON as a lightweight data format.

02

Explore popular libraries like **Gson**, **Jackson**, and **JSON.simple**.

03

Develop structured methods for reading/writing JSON safely and efficiently.

04

Apply JSON for configuration, APIs, and data exchange.

CONTENTS

1. Overview
2. JSON Data Format
3. Java JSON Libraries
4. Workflow of Reading and Writing JSON
5. Practices
6. Summary

1. OVERVIEW

JSON stands for **JavaScript Object Notation** which is a lightweight text-based open standard designed which is easy for human-readable data interchange. In general, JSON is extended from JavaScript. JSON is language-independent and It is easy to read and write. The file extension of JSON is `.json`.

JSON is a text-based data format that represents data in key - value pairs. In Java, JSON data is typically represented as strings. Before working with JSON, you need to understand the basic JSON data types:

- **Object:** A collection of key - value pairs, enclosed in curly braces `{}`.
- **Array:** An ordered list of values, enclosed in square brackets `[]`.
- **String:** A sequence of characters, enclosed in double quotes `"`.
- **Number:** A numeric value, either integer or floating - point.
- **Boolean:** Either `true` or `false`.
- **Null:** Represents the absence of a value, written as `null`.

2. JSON DATA FORMAT

Below are some of the standard JSON structures:

1. Basic JSON Strings

```
{  
  "name": "john"  
}
```

2. JSON Object

```
{  
  "name": "john",  
  "age": 42,  
  "skills": ["react", "java"],  
  "active": true  
}
```

3. JSON Array

```
[  
  {  
    "name": "john",  
    "age": 20  
  },  
  {  
    "name": "ah pig",  
    "age": 40  
  },  
  {  
    "name": "ah dog",  
    "age": 30  
  }  
]
```


3. JAVA JSON LIBRARIES

There are several popular JSON libraries in Java, each with its own features and advantages:

- **Jackson**: A high - performance JSON processing library that offers easy-to-use APIs for parsing and generating JSON.
- **Gson**: A simple library developed by Google for converting Java objects to JSON and vice versa.
- **JSON.org**: A basic library that provides simple classes for working with JSON in Java.
- **JSON.simple**: A lightweight utility for deserializing and serializing JSON.

In this slide, I'll focus on **Jackson** and **Gson**.

3. JAVA JSON LIBRARIES

Features	Jackson	Gson
Creator	FasterXML	Google
Simplicity	More advanced features	Very simple to use
Performance	Excellent	Good
Formatting	Pretty print + Streaming API	Pretty print support
Annotations	@JsonProperty, @JsonIgnore	@Expose, @SerializedName

3.1. JACKSON LIBRARY

Jackson is a powerful JSON library by **FasterXML**. It is widely used in enterprise-level applications and Spring Boot.

It provides:

- High performance
- Strong type safety
- Extensive annotations for control
- Streaming and data-binding APIs

Core Class: `com.fasterxml.jackson.databind.ObjectMapper`

3.1. JACKSON LIBRARY

1. Writing JSON (Serialization)

```
writeValue(File resultFile, Object value)
```

- Serializes a Java object into JSON and writes it to a file or stream.
- Converts complex objects, lists, and maps automatically.

Explanation:

- Jackson uses reflection to inspect Java fields and convert them to JSON keys.
- Fields can be controlled using annotations like:
 - `@JsonProperty("customName")` to rename a field.
 - `@JsonIgnore` to skip fields.
- `writerWithDefaultPrettyPrinter()` adds indentation for readability.

3.1. JACKSON LIBRARY

2. Reading JSON (Deserialization)

```
readValue(File src, Class<T> valueType)
```

- Converts JSON content from a file or stream into a Java object of type **T**.
- Matches JSON keys to Java field names automatically.

Explanation:

- The **ObjectMapper** creates an instance of the specified class and fills it with data from JSON.
- Nested objects and collections are deserialized recursively.
- Type mismatches or missing fields cause **MismatchedInputException**.

3.1. JACKSON LIBRARY

3. Supporting Methods

Methods	Description
<code>ObjectMapper()</code>	Core class for both read and write operations.
<code>writerWithDefaultPrettyPrinter()</code>	Produces human-readable JSON output.
<code>readTree()</code>	Parses JSON into a tree structure (JsonNode) for flexible traversal.
<code>convertValue()</code>	Converts one Java object type to another using JSON mapping internally.

3.2. GSON LIBRARY

Gson is a library developed by **Google** for converting Java objects to JSON and vice versa.

It supports:

- Automatic serialization and deserialization
- Pretty printing
- Handling of generic types
- Custom field naming using annotations

Main Class: [com.google.gson.Gson](#)

Builder Class: [GsonBuilder](#)

3.2. GSON LIBRARY

1. Writing JSON (Serialization)

```
toJson(Object src, Appendable writer)
```

- Converts a Java object into its JSON representation.
- Can write directly to a file, output stream, or string.
- The method automatically maps Java fields to JSON keys.

Explanation:

- Gson inspects each field of the object using reflection.
- Primitive types, Strings, arrays, collections, and objects are recursively converted into JSON.
- Formatting can be controlled using `GsonBuilder().setPrettyPrinting()`.

3.2. GSON LIBRARY

2. Reading JSON (Deserialization)

```
fromJson(Reader json, Class<T> classOfT)
```

- Converts a JSON string or file back into a Java object of type `T`.
- Maps JSON keys to corresponding Java fields automatically.

Explanation:

- Gson parses the JSON structure and creates an instance of the specified class.
- Field names must match the JSON keys (unless custom mapping is used).
- A **no-argument constructor** is required for the target class.

3.2. GSON LIBRARY

3. Supporting Methods

Methods	Description
<code>GsonBuilder()</code>	Configures a Gson instance (e.g., pretty printing).
<code>setPrettyPrinting()</code>	Produces human-readable formatted JSON output.
<code>toJsonTree()</code>	Converts an object to a <code>JsonElement</code> (useful for custom manipulation).
<code>fromJson(JsonElement, Class<T>)</code>	Parses from a JSON tree instead of raw text.

4. WORKFLOW OF READING AND WRITING JSON

Common Requirements

To successfully perform JSON read/write:

- The class must have a **public no-argument constructor**.
- All fields should have **getters** and **setters**.
- Field names must match JSON keys (unless customized).
- Proper exception handling is essential ([IOException](#), [JsonSyntaxException](#), [MismatchedInputException](#)).
- Always verify the file path when reading or writing.

4. WORKFLOW OF READING AND WRITING JSON

Serialization Workflow

1. Create a Java object.
2. Initialize a `Gson` or `ObjectMapper` instance.
3. Call the serialization method (`toJson()` or `writeValue()`).
4. JSON file or string is produced.

Deserialization Workflow

1. Prepare a JSON file or text input.
2. Initialize a `Gson` or `ObjectMapper` instance.
3. Call the deserialization method (`fromJson()` or `readValue()`).
4. Java object is reconstructed from JSON data.

5. PRACTICES

1. Setup Project with Gradle

Add dependencies in `build.gradle`:

```
dependencies {  
    implementation 'com.fasterxml.jackson.core:jackson-databind:2.20.1'  
    implementation 'com.google.code.gson:gson:2.13.2'  
}
```

Place your `data.json` in: `src/main/resources/data.json`

2. Run the project: `./gradlew.bat :app:run` (Windows) or `./gradlew :app:run` (MacOS)

5. PRACTICES

```
package org.example.demo.models;

public class User {
    private String name;
    private int age;
    private String email;

    public User() {
    }

    public User(String name, int age, String email) {
        this.name = name;
        this.age = age;
        this.email = email;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    public String getEmail() {
        return email;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setAge(int age) {
        this.age = age;
    }

    public void setEmail(String email) {
        this.email = email;
    }
}
```

5. PRACTICES

```
package org.example.demo.utils;

import java.io.IOException;
import java.io.Reader;
import java.nio.file.Files;
import java.nio.file.Path;

import org.example.demo.models.User;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.google.gson.Gson;

public class ReadJSON {
    public static void readUserDataGson(Path filePath) {
        Gson gson = new Gson();

        try (Reader reader = Files.newBufferedReader(filePath)) {
            User user = gson.fromJson(reader, User.class);
            System.out.println("(Gson) JSON data read successfully");
            printUser(user);
        } catch (IOException e) {
            System.err.println("Error reading JSON with Gson: " + e.getMessage());
        }
    }

    public static void readUserDataJackson(Path filePath) {
        ObjectMapper mapper = new ObjectMapper();

        try {
            User user = mapper.readValue(filePath.toFile(), User.class);
            System.out.println("(Jackson) JSON data read successfully");
            printUser(user);
        } catch (IOException e) {
            System.err.println("Error reading JSON with Jackson: " + e.getMessage());
        }
    }

    private static void printUser(User user) {
        if (user != null) {
            System.out.println("Name: " + user.getName());
            System.out.println("Age: " + user.getAge());
            System.out.println("Email: " + user.getEmail());
        } else {
            System.out.println("No user data found");
        }
    }
}
```

5. PRACTICES

```
package org.example.demo.utils;

import java.io.IOException;
import java.io.Writer;
import java.nio.file.Files;
import java.nio.file.Path;

import org.example.demo.models.User;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.google.gson.Gson;
import com.google.gson.GsonBuilder;

public class WriteJSON {
    public static void writeUserDataGson(Path filePath, User user) {
        Gson gson = new GsonBuilder().setPrettyPrinting().create();

        try (Writer writer = Files.newBufferedWriter(filePath)) {
            gson.toJson(user, writer);
            System.out.println("(Gson) JSON data written successfully to " + filePath);
        } catch (IOException e) {
            System.err.println("Error writing JSON with Gson: " + e.getMessage());
        }
    }

    public static void writeUserDataJackson(Path filePath, User user) {
        ObjectMapper mapper = new ObjectMapper();

        try {
            mapper.writerWithDefaultPrettyPrinter().writeValue(filePath.toFile(), user);
            System.out.println("(Jackson) JSON data written successfully to " + filePath);
        } catch (IOException e) {
            System.err.println("Error writing JSON with Jackson: " + e.getMessage());
        }
    }
}
```


5. PRACTICES

```
package org.example.demo;

import java.nio.file.Path;
import java.nio.file.Paths;

import org.example.demo.models.User;
import org.example.demo.utils.ReadJSON;
import org.example.demo.utils.WriteJSON;

public class DemoApplication {
    private static final Path FILE_PATH = Paths.get("src/main/resources/data.json");

    public static void main(String[] args) {
        User user1 = new User("Alice", 25, "alice@example.com");
        WriteJSON.writeUserDataGson(FILE_PATH, user1);
        ReadJSON.readUserDataGson(FILE_PATH);

        User user2 = new User("Bob", 30, "bob@example.com");
        WriteJSON.writeUserDataJackson(FILE_PATH, user2);
        ReadJSON.readUserDataJackson(FILE_PATH);
    }
}
```

Output:

(Gson) JSON data written successfully to
src/main/resources/data.json

(Gson) JSON data read successfully

Name: Alice

Age: 25

Email: alice@example.com

(Jackson) JSON data written successfully
to src/main/resources/data.json

(Jackson) JSON data read successfully

Name: Bob

Age: 30

Email: bob@example.com

6. SUMMARY

Working with JSON in Java is a common task, and there are several reliable libraries available. **Jackson** and **Gson** are two popular choices that offer easy-to-use APIs for parsing, generating, and mapping JSON data. By understanding the fundamental concepts, usage methods, common practices, and best practices, you can efficiently work with JSON in your Java applications.